torch.sparse.mm#

https://pytorch.org/docs/stable/generated/torch.sparse.mm.html

Description:

Performs a matrix multiplication of the sparse matrix mat1 and the (sparse or strided) matrix mat2. Similar to torch.mm(), if mat1 is a (n×m)(n \times m)(n×m) tensor, mat2 is a (m×p)(m \times p)(m×p) tensor, out will be a (n×p)(n \times p)(n×p) tensor. When mat1 is a COO tensor it must have sparse_dim = 2. When inputs are COO tensors, this function also supports backward for both inputs. Supports both CSR and COO storage formats. This function doesn't support computing derivatives with respect to CSR matrices. This function also additionally accepts an optional reduce argument that allows specification of an optional reduction operation, mathematically performs the following operation:

Function Signature

```
torch.sparse.mm()#
Parameters
Shape:
```

Code Examples

```
>>> a = torch.tensor([[1., 0, 2], [0, 3,
0]]).to_sparse().requires_grad_()
>>> a
tensor(indices=tensor([[0, 0, 1],
                       [0, 2, 1]]),
       values=tensor([1., 2., 3.]),
       size=(2, 3), nnz=3, layout=torch.sparse_coo,
requires_grad=True)
>>> b = torch.tensor([[0, 1.], [2, 0], [0, 0]],
requires_grad=True)
>>> b
tensor([[0., 1.],
        [2., 0.],
        [0., 0.]], requires_grad=True)
>>> y = torch.sparse.mm(a, b)
>>> y
tensor([[0., 1.],
        [6., 0.]], grad_fn=<SparseAddmmBackward0>)
>>> y.sum().backward()
>>> a.grad
tensor(indices=tensor([[0, 0, 1],
                       [0, 2, 1]]),
       values=tensor([1., 0., 2.]),
       size=(2, 3), nnz=3, layout=torch.sparse_coo)
>>> c = a.detach().to_sparse_csr()
>>> c
tensor(crow_indices=tensor([0, 2, 3]),
       col_indices=tensor([0, 2, 1]),
       values=tensor([1., 2., 3.]), size=(2, 3), nnz=3,
       layout=torch.sparse_csr)
>>> y1 = torch.sparse.mm(c, b, 'sum')
>>> y1
tensor([[0., 1.],
```

```
        [6., 0.]], grad_fn=<SparseMmReduceImplBackward0>)
>>> y2 = torch.sparse.mm(c, b, 'max')
>>> y2
tensor([[0., 1.],
        [6., 0.]], grad_fn=<SparseMmReduceImplBackward0>)
```

Notes & Warnings

NOTE

Note This function doesn't support computing derivatives with respect to CSR matrices. This function also additionally accepts an optional reduce argument that allows specification of an optional reduction operation, mathematically performs the following operation:

Detailed Documentation

Documentation

Performs a matrix multiplication of the sparse matrix mat1 and the (sparse or strided) matrix mat2. Similar to torch.mm(), if mat1 is a (n×m)(n \times m)(n×m) tensor, mat2 is a (m×p)(m \times p)(m×p) tensor, out will be a (n×p)(n \times p)(n×p) tensor. When mat1 is a COO tensor it must have sparse_dim = 2. When inputs are COO tensors, this function also supports backward for both inputs.

Supports both CSR and COO storage formats.

This function doesn’t support computing derivatives with respect to CSR matrices.

This function also additionally accepts an optional reduce argument that allows specification of an optional reduction operation, mathematically performs the following operation:

where ⊕\bigoplus⊕ defines the reduce operator. reduce is implemented only for CSR storage format on CPU device.

mat1 (Tensor) – the first sparse matrix to be multiplied

mat2 (Tensor) – the second matrix to be multiplied, which could be sparse or dense

reduce (str, optional) – the reduction operation to apply for non-unique indices ("sum", "mean", "amax", "amin"). Default "sum".

The format of the output tensor of this function follows: - sparse x sparse -> sparse - sparse x dense -> dense